

# HomeWork5

Aswath Oruganti

July 16, 2026

## 1 Concept Questions:

1. Variance of Correlated Averages:

### 1.1 Mathematical Derivation:

**Objective:** Show that the variance of the average of  $B$  identically distributed (i.d.) random variables with variance  $\sigma^2$  and pairwise correlation  $\rho$  is given by:

$$\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

**Given:**

1. A set of  $B$  random variables:  $X_1, X_2, \dots, X_B$ .
2. The variables are identically distributed, meaning they share the same variance:  $\text{Var}(X_i) = \sigma^2$  for all  $i$ .
3. The pairwise correlation between any two distinct variables  $X_i$  and  $X_j$  (where  $i \neq j$ ) is  $\rho = \frac{\text{Cov}(X_i, X_j)}{\sigma^2}$ .  
*By rearranging this, we know that:  $\text{Cov}(X_i, X_j) = \rho\sigma^2$ .*
4. The sample average is defined as:  $\bar{X} = \frac{1}{B} \sum_{i=1}^B X_i$ .

**Step-by-Step Proof:**

**Step 1: Apply the variance operator to the sample average.**

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{B} \sum_{i=1}^B X_i\right)$$

**Step 2: Pull the constant out of the variance.** Recall the property of variance that  $\text{Var}(cX) = c^2\text{Var}(X)$ . Here, our constant  $c$  is  $\frac{1}{B}$ .

$$\text{Var}(\bar{X}) = \frac{1}{B^2} \text{Var}\left(\sum_{i=1}^B X_i\right)$$

**Step 3: Expand the variance of the sum.** The variance of a sum of random variables is equal to the sum of their individual variances plus the sum of all their pairwise covariances.

$$\text{Var}(\bar{X}) = \frac{1}{B^2} \left[ \sum_{i=1}^B \text{Var}(X_i) + \sum_{i=1}^B \sum_{j \neq i}^B \text{Cov}(X_i, X_j) \right]$$

**Step 4: Substitute the known values and count the terms.**

- **Variance terms:** There are exactly  $B$  variables, and each has a variance of  $\sigma^2$ . So,  $\sum_{i=1}^B \text{Var}(X_i) = B\sigma^2$ .
- **Covariance terms:** Out of a total  $B \times B$  possible pairs in a covariance matrix, there are  $B$  diagonal terms (the variances we already counted). This leaves exactly  $B^2 - B = B(B - 1)$  off-diagonal terms. Every single one of these pairs has a covariance of  $\rho\sigma^2$ . So,  $\sum_{i=1}^B \sum_{j \neq i}^B \text{Cov}(X_i, X_j) = B(B - 1)\rho\sigma^2$ .

Substitute these back into the equation:

$$\text{Var}(\bar{X}) = \frac{1}{B^2} [B\sigma^2 + B(B - 1)\rho\sigma^2]$$

**Step 5: Distribute and simplify.** Divide both terms inside the bracket by  $B^2$ :

$$\begin{aligned}\text{Var}(\bar{X}) &= \frac{B\sigma^2}{B^2} + \frac{B(B - 1)\rho\sigma^2}{B^2} \\ \text{Var}(\bar{X}) &= \frac{\sigma^2}{B} + \frac{(B - 1)}{B}\rho\sigma^2\end{aligned}$$

**Step 6: Rearrange into the target form.** Split the second term:

$$\begin{aligned}\text{Var}(\bar{X}) &= \frac{\sigma^2}{B} + \left(\frac{B}{B} - \frac{1}{B}\right)\rho\sigma^2 \\ \text{Var}(\bar{X}) &= \frac{\sigma^2}{B} + \rho\sigma^2 - \frac{\rho\sigma^2}{B}\end{aligned}$$

Group the terms divided by  $B$  together:

$$\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{\sigma^2 - \rho\sigma^2}{B}$$

Factor out  $\sigma^2$  from the numerator of the second fraction:

$$\text{Var}(\bar{X}) = \rho\sigma^2 + \frac{1 - \rho}{B}\sigma^2$$

(Q.E.D.)

## 2. Why this is important for Random Forests:

In the context of ensemble learning,  $X_i$  represents the prediction of a single decision tree, and  $\bar{X}$  is the final averaged prediction of the entire ensemble (a forest of  $B$  trees). The goal of ensemble methods like Bagging and Random Forest is to reduce the variance of the model to prevent overfitting. Looking closely at the derived formula:

$$\text{Var}(\text{Ensemble}) = \rho\sigma^2 + \frac{1 - \rho}{B}\sigma^2$$

We can see exactly why Random Forest was invented and why decorrelating trees is crucial:

1. **The Variance Bottleneck ( $\rho\sigma^2$ ):** As you increase the number of trees in your forest ( $B \rightarrow \infty$ ), the second term ( $\frac{1 - \rho}{B}\sigma^2$ ) shrinks to zero. However, the first term ( $\rho\sigma^2$ ) remains completely unchanged. No matter how many trees you add, **the variance of your forest can never drop below  $\rho\sigma^2$ .**
2. **The Problem with Standard Bagging:** In standard Bagging, every tree has access to all the same features in the dataset. If there is one incredibly strong predictor feature, almost every single tree will use that feature for its first split. Because the trees look highly similar, their predictions are highly correlated ( $\rho$  is close to 1). If  $\rho \approx 1$ , the bottleneck  $\rho\sigma^2$  is massive, and averaging the trees barely reduces the variance at all.

3. **The Random Forest Solution (Decorrelation):** Random Forest explicitly forces the trees to be different by randomly restricting the subset of features available at each split (feature bagging). This artificially “blinds” trees to dominant predictors, forcing them to learn different patterns. By doing this, **Random Forest artificially drives down  $\rho$  (the correlation between trees)**. While forcing trees to use suboptimal splits slightly increases the variance ( $\sigma^2$ ) and bias of an individual tree, the massive reduction in  $\rho$  drastically shrinks the  $\rho\sigma^2$  bottleneck. This allows the overall ensemble to achieve a vastly lower variance than standard Bagging, leading to a much more robust and accurate model.

## 2 The Bayesian Classifier Minimizes Classification Risk:

### 1. Problem Statement:

Let  $X \in \mathbb{R}^d$  be a feature vector, and  $Y \in \{1, 2, \dots, K\}$  be the class label. Let  $P(Y = k | X)$  be the posterior probability of class  $k$  given the feature  $X$ . Let  $h(X)$  be any classifier mapping  $X$  to one of the class labels. Define the risk of the classifier as the probability of misclassification:

$$R(h) = P(h(X) \neq Y)$$

Show that the Bayesian classifier  $h^*$  (defined by picking the maximum posterior) achieves the minimum risk among all classifiers:

$$h^* = \arg \min_h R(h)$$

#### 1.1 Proof:

##### Step 1: Relate Risk to Probability of Correct Classification

Minimizing the probability of being wrong is mathematically identical to maximizing the probability of being correct. We can rewrite the risk as:

$$R(h) = 1 - P(h(X) = Y)$$

Therefore, finding the optimal classifier  $h^*$  that minimizes  $R(h)$  is equivalent to finding the  $h$  that maximizes  $P(h(X) = Y)$ .

##### Step 2: Expand the Probability using Total Expectation

Using the Law of Total Expectation, we can express the overall probability of a correct classification in terms of the conditional expectation given the features  $X$ :

$$P(h(X) = Y) = \mathbb{E}_X[\mathbb{E}_{Y|X}[I(h(X) = Y) | X]]$$

where  $I(\cdot)$  is the indicator function, which equals 1 if the condition is true and 0 otherwise.

##### Step 3: Evaluate the Inner Expectation

For a specific, fixed observation  $X = x$ , the expectation of the indicator function is simply the probability of the event it indicates:

$$\mathbb{E}_{Y|X}[I(h(x) = Y) | X = x] = P(Y = h(x) | X = x)$$

This term represents the posterior probability that the true class is whatever our classifier  $h(x)$  chooses, given the specific data point  $x$ .

#### Step 4: Substitute into the Outer Expectation

Substituting the result from Step 3 back into our original expectation equation gives:

$$P(h(X) = Y) = \mathbb{E}_X[P(Y = h(X) \mid X)]$$

If  $X$  is a continuous random variable with a probability density function  $p(x)$ , this expectation is an integral over the entire feature space:

$$P(h(X) = Y) = \int_{\mathcal{X}} P(Y = h(x) \mid X = x) p(x) dx$$

#### Step 5: Pointwise Maximization

Our objective is to maximize this entire expectation (or integral). Because probabilities and probability density functions are strictly non-negative ( $p(x) \geq 0$ ), the only way to maximize the total expected value is to maximize the inner term for *every single value* of  $x$ . Therefore, for any given input  $x$ , we must choose the class prediction  $h(x)$  that maximizes the term  $P(Y = h(x) \mid X = x)$ .

#### Step 6: Define the Optimal Classifier

To achieve this pointwise maximum, our optimal classifier  $h^*(x)$  must be defined as the class  $k$  that yields the highest posterior probability:

$$h^*(x) = \arg \max_{k \in \{1, 2, \dots, K\}} P(Y = k \mid X = x)$$

#### 2. Conclusion:

Because the Bayesian classifier  $h^*(x)$  explicitly picks the maximum posterior probability for every possible  $X$ , it maximizes the total expected value  $\mathbb{E}_X[P(Y = h(X) \mid X)]$ . By maximizing  $P(h(X) = Y)$ , it strictly minimizes the risk  $R(h) = 1 - P(h(X) = Y)$ . Thus:

$$h^* = \arg \min_h R(h)$$

### 3 Medical Imaging Reconstruction (Compressed Sensing):

Below is the complete solution, mathematical breakdown, and Python implementation for the medical imaging reconstruction assignment. I have provided the code snippets and corresponding visualizations at the relevant sections.

#### 1. Data Generation and Setup:

The problem states that we have a sparse image  $\mathbf{x} \in \mathbb{R}^{2500}$  ( $50 \times 50$  pixels) where only 416 pixels are non-zero. The measurements are taken via a linear combination mapping:

$$\mathbf{y} = A\mathbf{x} + \boldsymbol{\epsilon}$$

Where:

- $A \in \mathbb{R}^{1300 \times 2500}$  with entries  $A_{ij} \sim \mathcal{N}(0, 1)$ .
- $\boldsymbol{\epsilon} \sim \mathcal{N}(0, 25I_{1300})$ . Note that the variance is 25, meaning the standard deviation is  $\sqrt{25} = 5$ .
- $\mathbf{y} \in \mathbb{R}^{1300}$  is our observed measurement vector.

## 1.1 Python Implementation: Setup

```
1 import numpy as np
2 import scipy.io
3 import matplotlib.pyplot as plt
4
5 # Load the MATLAB file
6 mat = scipy.io.loadmat('cs.mat')
7 var_name = [k for k in mat.keys() if not k.startswith('__')][0]
8 img_true = mat[var_name]
9
10 # Flatten the 50x50 image into a 2500x1 vector
11 x_true = img_true.flatten()
12 p = len(x_true) # 2500
13 n = 1300        # number of measurements
14
15 np.random.seed(42)
16
17 # Generate measurement matrix A and noisy observations y
18 A = np.random.randn(n, p)
19 eps = np.random.normal(loc=0.0, scale=5.0, size=n)
20 y = A @ x_true + eps
```

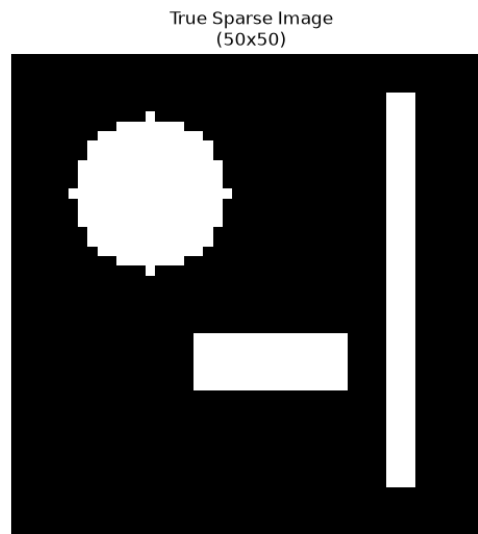


Figure 1: The original, true sparse image to be reconstructed.

## 2. LASSO Reconstruction with L1 Penalty (10-Fold Cross-Validation):

Lasso regression optimizes the objective:

$$\min_{\mathbf{x}} \|\mathbf{y} - A\mathbf{x}\|_2^2 + \lambda \|\mathbf{x}\|_1$$

By applying an L1 penalty, Lasso enforces **sparsity**. It forces many coefficients to become exactly zero. Because we know our true image is highly sparse (2084 zero pixels), Lasso is theoretically perfectly suited for this compressed sensing task. Using 10-fold cross-validation, the algorithm searches for the optimal regularization strength  $\lambda$  that minimizes the Mean Squared Error (MSE).

## 2.1 Python Implementation: Lasso

We use `LassoCV(cv=10)`, which is highly optimized for L1 regularization. Calling `.fit(A, y)` automatically generates an array of 100 different  $\lambda$  values, tests every  $\lambda$  on every fold, and computes the average MSE. Once the optimal  $\lambda$  is found, it refits a final model on the entire dataset.

```
1 from sklearn.linear_model import LassoCV
2
3 # Run Lasso with 10-fold CV
4 lasso_cv = LassoCV(cv=10, max_iter=10000, random_state=42, n_jobs=-1)
5 lasso_cv.fit(A, y)
6
7 best_alpha_lasso = lasso_cv.alpha_
8 x_lasso = lasso_cv.coef_
9 img_lasso = x_lasso.reshape(50, 50)
10
11 # Extract Cross-Validation errors for plotting
12 m_log_alphas_lasso = np.log10(lasso_cv.alphas_)
13 mean_mse_lasso = lasso_cv.mse_path_.mean(axis=-1)
```

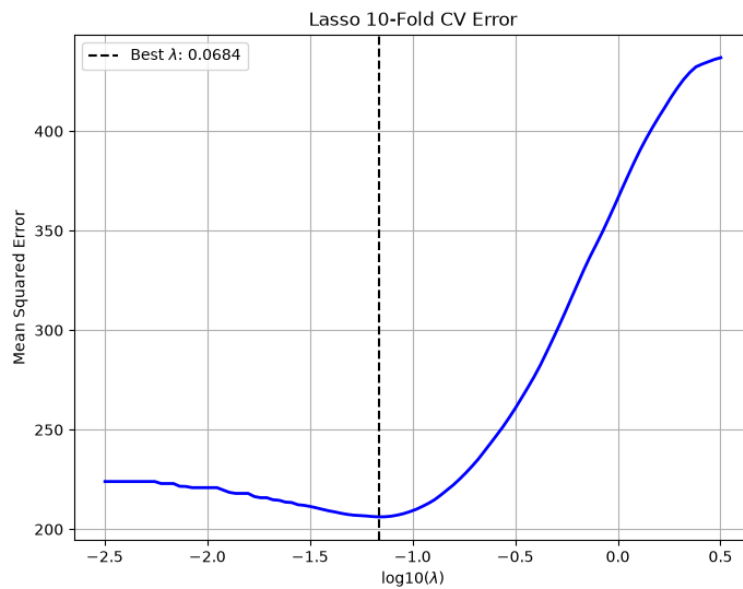


Figure 2: Lasso 10-Fold CV Error Curve. The dashed line represents the optimal  $\lambda$ .

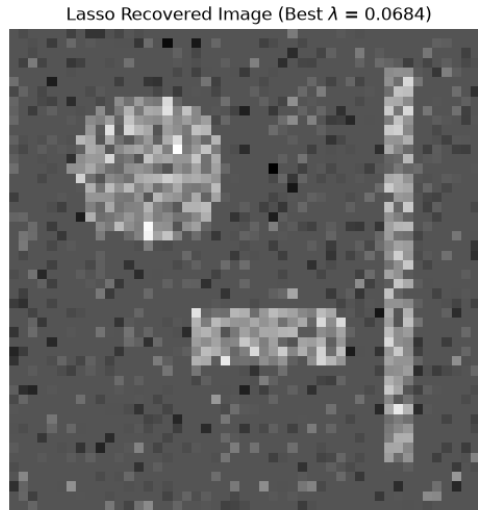


Figure 3: Lasso Recovered Image using the optimal  $\lambda$ .

### 3. Ridge Reconstruction with L2 Penalty (10-Fold Cross-Validation):

Ridge regression optimizes the objective:

$$\min_{\mathbf{x}} \|\mathbf{y} - A\mathbf{x}\|_2^2 + \lambda \|\mathbf{x}\|_2^2$$

An L2 penalty shrinks the coefficients toward zero but **rarely makes them exactly zero**. It tends to distribute the weights across all features. Because it cannot effectively zero-out the background pixels, the Ridge reconstruction will suffer from high background noise.

#### 3.1 Python Implementation: Ridge

We manually define a logarithmic range of  $\lambda$  values using `np.logspace`. We then wrap a standard `Ridge()` model inside a `GridSearchCV` object, instructing it to test our defined grid using 10 folds and optimize for negative MSE.

```

1 from sklearn.linear_model import Ridge
2 from sklearn.model_selection import GridSearchCV
3
4 # Define a grid of alphas for Ridge to search through
5 alphas_ridge = np.logspace(-1, 5, 100)
6 ridge_model = Ridge(max_iter=10000)
7
8 ridge_grid = GridSearchCV(ridge_model, param_grid={'alpha': alphas_ridge},
9                           cv=10, scoring='neg_mean_squared_error', n_jobs=-1)
10 ridge_grid.fit(A, y)
11
12 best_alpha_ridge = ridge_grid.best_params_['alpha']
13 x_ridge = ridge_grid.best_estimator_.coef_
14 img_ridge = x_ridge.reshape(50, 50)
15
16 # Extract Cross-Validation errors for plotting
17 m_log_alphas_ridge = np.log10(alphas_ridge)
18 mean_mse_ridge = -ridge_grid.cv_results_['mean_test_score']

```

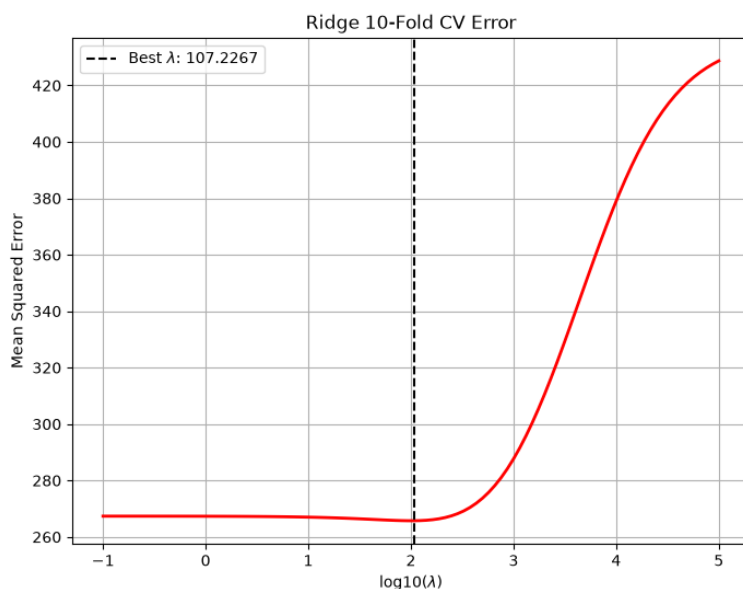


Figure 4: Ridge 10-Fold CV Error Curve. The dashed line represents the optimal  $\lambda$ .

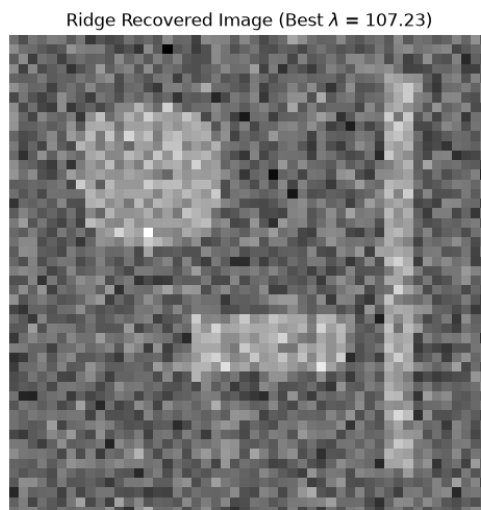


Figure 5: Ridge Recovered Image using the optimal  $\lambda$ . Notice the high background noise compared to Lasso.

#### 4. Comparison Conclusion: Which approach is better?

**Lasso gives a significantly better recovered image.** Because the true underlying image is truly sparse (only 416 non-zero pixels out of 2500), the L1 penalty of Lasso actively exploits this structural assumption, zeroing out the empty space and producing a clean, sharp image. Ridge regression (L2), on the other hand, distributes the error and weights across all 2500 pixels, resulting in a blurry, static-filled reconstruction that fails to isolate the true shape. This fundamental difference demonstrates the core power of **Compressed**

**Sensing:** you can recover a high-dimensional signal from fewer measurements ( $n < p$ ) if the signal is sparse and you use an L1-based recovery method.

## 4 AdaBoost Binary Classification:

### 1. AdaBoost Decision Stump—3 Iterations:

#### 1.1 Step-by-Step AdaBoost Execution ( $T = 3$ )

##### Initial Setup:

We have  $m = 8$  data points. The dataset is:

- $X_1 = (-1, 0), y_1 = +1$
- $X_2 = (-0.5, 0.5), y_2 = +1$
- $X_3 = (0, 1), y_3 = -1$
- $X_4 = (0.5, 1), y_4 = -1$
- $X_5 = (1, 0), y_5 = +1$
- $X_6 = (1, -1), y_6 = +1$
- $X_7 = (0, -1), y_7 = -1$
- $X_8 = (0, 0), y_8 = -1$

At  $t = 1$ , the weights are initialized uniformly:

$$D_1(i) = \frac{1}{m} = \frac{1}{8} = 0.125 \quad \text{for } i = 1, \dots, 8$$

#### 1.2 Iteration $t = 1$ :

##### 1. Find the best weak learner $h_1(x)$ :

We choose a vertical split on the  $x^{(1)}$ -axis:

$$h_1(x) = \begin{cases} +1 & \text{if } x^{(1)} < -0.25 \\ -1 & \text{if } x^{(1)} > -0.25 \end{cases}$$

Predictions: Correct for  $X_1, X_2, X_3, X_4, X_7, X_8$ . Incorrect for  $X_5, X_6$ .

Misclassified points:  $X_5, X_6$ .

##### 2. Compute Error ( $\epsilon_1$ ):

$$\epsilon_1 = D_1(5) + D_1(6) = \frac{1}{8} + \frac{1}{8} = \frac{1}{4} = 0.25$$

##### 3. Compute Model Weight ( $\alpha_1$ ):

$$\alpha_1 = \frac{1}{2} \ln \left( \frac{1 - \epsilon_1}{\epsilon_1} \right) = \frac{1}{2} \ln \left( \frac{0.75}{0.25} \right) = \frac{1}{2} \ln(3) \approx 0.5493$$

**4. Compute Normalization Factor ( $Z_1$ ):**

$$Z_1 = 2\sqrt{\epsilon_1(1 - \epsilon_1)} = 2\sqrt{0.25 \times 0.75} = \frac{\sqrt{3}}{2} \approx 0.8660$$

**5. Update Weights ( $D_2$ ):**

$$D_2(i) = \frac{D_1(i) \exp(-\alpha_1 y_i h_1(x_i))}{Z_1}$$

- Correctly classified ( $i \in \{1, 2, 3, 4, 7, 8\}$ ):  $D_2(i) = \frac{(1/8)(1/\sqrt{3})}{\sqrt{3}/2} = \frac{1}{12}$
- Incorrectly classified ( $i \in \{5, 6\}$ ):  $D_2(i) = \frac{(1/8)(\sqrt{3})}{\sqrt{3}/2} = \frac{1}{4}$

**1.3 Iteration  $t = 2$ :**

**1. Find the best weak learner  $h_2(x)$ :**

$X_5$  and  $X_6$  now hold heavy weights ( $1/4$  each). We find a stump that classifies them correctly:

$$h_2(x) = \begin{cases} +1 & \text{if } x^{(1)} > 0.75 \\ -1 & \text{if } x^{(1)} < 0.75 \end{cases}$$

Predictions: Correct for  $X_3, X_4, X_5, X_6, X_7, X_8$ . Incorrect for  $X_1, X_2$ .  
Misclassified points:  $X_1, X_2$ .

**2. Compute Error ( $\epsilon_2$ ):**

$$\epsilon_2 = D_2(1) + D_2(2) = \frac{1}{12} + \frac{1}{12} = \frac{1}{6} \approx 0.1667$$

**3. Compute Model Weight ( $\alpha_2$ ):**

$$\alpha_2 = \frac{1}{2} \ln \left( \frac{1 - 1/6}{1/6} \right) = \frac{1}{2} \ln(5) \approx 0.8047$$

**4. Compute Normalization Factor ( $Z_2$ ):**

$$Z_2 = 2\sqrt{\frac{1}{6} \times \frac{5}{6}} = \frac{\sqrt{5}}{3} \approx 0.7454$$

**5. Update Weights ( $D_3$ ):**

- Correctly classified ( $i \in \{3, 4, 7, 8\}$ ):  $D_3(i) = \frac{1}{12} \times \frac{3}{5} = \frac{1}{20}$
- Correctly classified ( $i \in \{5, 6\}$ ):  $D_3(i) = \frac{1}{4} \times \frac{3}{5} = \frac{3}{20}$
- Incorrectly classified ( $i \in \{1, 2\}$ ):  $D_3(i) = \frac{1}{12} \times 3 = \frac{1}{4}$

#### 1.4 Iteration $t = 3$ :

##### 1. Find the best weak learner $h_3(x)$ :

$X_1, X_2$  are heavily weighted ( $1/4$ ), and  $X_5, X_6$  are  $3/20$ . A horizontal split classifies these  $+1$  points correctly:

$$h_3(x) = \begin{cases} +1 & \text{if } x^{(2)} < 0.75 \\ -1 & \text{if } x^{(2)} > 0.75 \end{cases}$$

Predictions: Correct for  $X_1, X_2, X_3, X_4, X_5, X_6$ . Incorrect for  $X_7, X_8$ .

Misclassified points:  $X_7, X_8$ .

##### 2. Compute Error ( $\epsilon_3$ ):

$$\epsilon_3 = D_3(7) + D_3(8) = \frac{1}{20} + \frac{1}{20} = \frac{1}{10} = 0.10$$

##### 3. Compute Model Weight ( $\alpha_3$ ):

$$\alpha_3 = \frac{1}{2} \ln \left( \frac{1 - 0.1}{0.1} \right) = \frac{1}{2} \ln(9) = \ln(3) \approx 1.0986$$

##### 4. Compute Normalization Factor ( $Z_3$ ):

$$Z_3 = 2\sqrt{0.1 \times 0.9} = 2(0.3) = 0.6$$

#### 1.5 Completed Parameters Table:

| $t$ | $\epsilon_t$ | $\alpha_t$                         | $Z_t$                              | $D_t(1)$ | $D_t(2)$ | $D_t(3)$ | $D_t(4)$ | $D_t(5)$ | $D_t(6)$ | $D_t(7)$ | $D_t(8)$ |
|-----|--------------|------------------------------------|------------------------------------|----------|----------|----------|----------|----------|----------|----------|----------|
| 1   | $1/4$        | $\frac{1}{2} \ln(3) \approx 0.549$ | $\frac{\sqrt{3}}{2} \approx 0.866$ | $1/8$    | $1/8$    | $1/8$    | $1/8$    | $1/8$    | $1/8$    | $1/8$    | $1/8$    |
| 2   | $1/6$        | $\frac{1}{2} \ln(5) \approx 0.805$ | $\frac{\sqrt{5}}{3} \approx 0.745$ | $1/12$   | $1/12$   | $1/12$   | $1/12$   | $1/4$    | $1/4$    | $1/12$   | $1/12$   |
| 3   | $1/10$       | $\ln(3) \approx 1.099$             | $0.6$                              | $1/4$    | $1/4$    | $1/20$   | $1/20$   | $3/20$   | $3/20$   | $1/20$   | $1/20$   |

## 2. Final AdaBoost Model Training Error

The final strong classifier is the sign of the weighted sum of our three weak learners:

$$H(x) = \text{sign}(\alpha_1 h_1(x) + \alpha_2 h_2(x) + \alpha_3 h_3(x))$$

$$H(x) = \text{sign}(0.549h_1(x) + 0.805h_2(x) + 1.099h_3(x))$$

Evaluating the training error on the points that were misclassified at various steps:

- **Check**  $X_1, X_2$  (Misclassified by  $h_2$ ):

$$h_1 = +1, h_2 = -1, h_3 = +1 \implies 0.549(1) - 0.805(1) + 1.099(1) = 0.843 > 0 \implies H(x) = +1 \text{ (Correct)}.$$

- **Check**  $X_5, X_6$  (Misclassified by  $h_1$ ):

$$h_1 = -1, h_2 = +1, h_3 = +1 \implies -0.549(1) + 0.805(1) + 1.099(1) = 1.355 > 0 \implies H(x) = +1 \text{ (Correct)}.$$

- **Check**  $X_7, X_8$  (Misclassified by  $h_3$ ):

$$h_1 = -1, h_2 = -1, h_3 = +1 \implies -0.549(1) - 0.805(1) + 1.099(1) = -0.255 < 0 \implies H(x) = -1 \text{ (Correct)}.$$

- Check  $X_3, X_4$  (Never misclassified):  
All negative  $\implies H(x) = -1$  (Correct).

The training error of this final AdaBoost model is 0.

## 2.1 Why AdaBoost outperforms a single decision stump:

A single decision stump can only split the feature space with a single axis-aligned line. In this dataset, the  $+1$  points lie on the far left and far right, while the  $-1$  points are clustered in the middle. It is mathematically impossible to draw a single straight line that perfectly separates the  $+1$ s from the  $-1$ s (a single stump will always have an error  $\geq 0.25$ ). AdaBoost overcomes this limitation by combining multiple weak learners. By iteratively up-weighting the data points that prior stumps failed to classify correctly, AdaBoost forces subsequent stumps to focus on those “hard” regions. The final ensemble model essentially stitches these lines together, creating a complex, non-linear decision boundary capable of perfectly separating the classes.

# 5 Random Forest and One-Class SVM for Email Spam Classification:

The following sections provide the construction, evaluation, and theoretical analysis of various machine learning models (CART, Random Forest, and One-Class SVM) designed to classify the UCI Spambase dataset.

## 1. Data Setup & Preprocessing

This section provide the first step of data loading and preparation. The UCI Spambase dataset consists of 57 numeric features (word frequencies, character frequencies, and capital run lengths) and one target label indicating whether the email is spam (1) or non-spam (0).

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from sklearn.tree import DecisionTreeClassifier, plot_tree
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.svm import OneClassSVM
8 from sklearn.metrics import accuracy_score
9 from sklearn.preprocessing import StandardScaler
10
11 # Load data and handle missing values
12 df = pd.read_csv('spambase.data', header=None)
13 df.fillna(0, inplace=True)
14
15 # Separate features (X) and target label (y)
16 X = df.iloc[:, :-1].values
17 y = df.iloc[:, -1].values
18
19 # Partition the dataset: 75% training, 25% testing
20 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
    random_state=42)

```

### 1.1 Code Explanation:

- `pd.read_csv(..., header=None)`: Loads the CSV file into a pandas DataFrame. `header=None` indicates the first row contains data, not column names, causing pandas to automatically assign numeric indices.

- `df.fillna(0, inplace=True)`: Searches the dataset for any missing or null values (NaN) and replaces them with 0 to ensure mathematical operations do not fail.
- `X = df.iloc[:, :-1].values`: Extracts the independent features by selecting all rows and all columns up to (but not including) the last column. `.values` converts it to a NumPy array.
- `y = df.iloc[:, -1].values`: Extracts the dependent target labels by selecting only the final column.

## 2. Classification and Regression Tree (CART):

The python implementation builds a fully grown CART model on the training data. As requested, pruning is not applied to the actual model being evaluated. However, for the sake of visualization legibility, the plotted diagram of the tree is truncated at a depth of 3. A fully grown tree on 57 features contains hundreds of nodes and is visually unreadable when plotted in its entirety.

```

1 # Build and evaluate the fully-grown CART model
2 dt_full = DecisionTreeClassifier(random_state=42)
3 dt_full.fit(X_train, y_train)
4
5 # Calculate test error rate
6 cart_test_error = 1 - accuracy_score(y_test, dt_full.predict(X_test))

```

**CART Test Error Rate: 0.0851 (8.51%)**

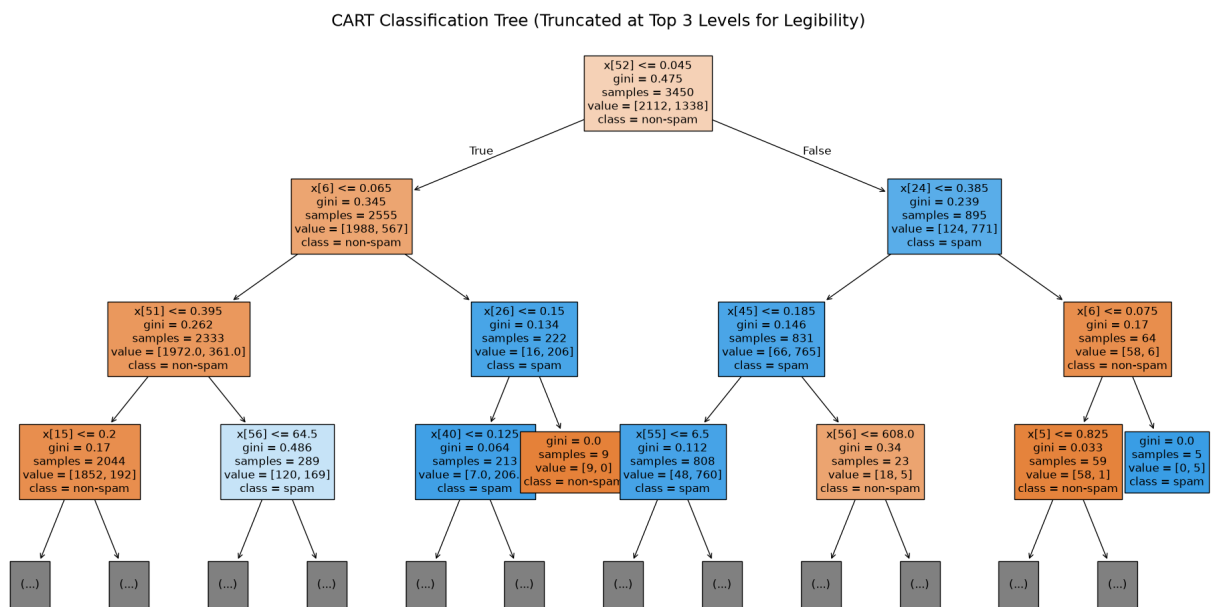


Figure 6: CART Classification Tree (Truncated at Top 3 Levels for Legibility)

## 3. Random Forest vs. CART:

In this section, we evaluate an ensemble Random Forest model. We systematically vary the number of trees (`n_estimators`) from 10 to 200 to observe how the ensemble size affects predictive performance.

```

1 n_trees = list(range(10, 210, 10))
2 rf_errors = []
3

```

```

4 for n in n_trees:
5     rf = RandomForestClassifier(n_estimators=n, random_state=42, n_jobs=-1)
6     rf.fit(X_train, y_train)
7     rf_errors.append(1 - accuracy_score(y_test, rf.predict(X_test)))

```

**Minimum RF Test Error Rate: 0.0400 (4.00%)** achieved at **30 trees**. As clearly seen in the plot below, the Random Forest drastically outperforms the single classification tree. By bagging the data and randomly sampling subsets of features at each split, the forest significantly reduces model variance and combats the overfitting inherent to single fully-grown decision trees.

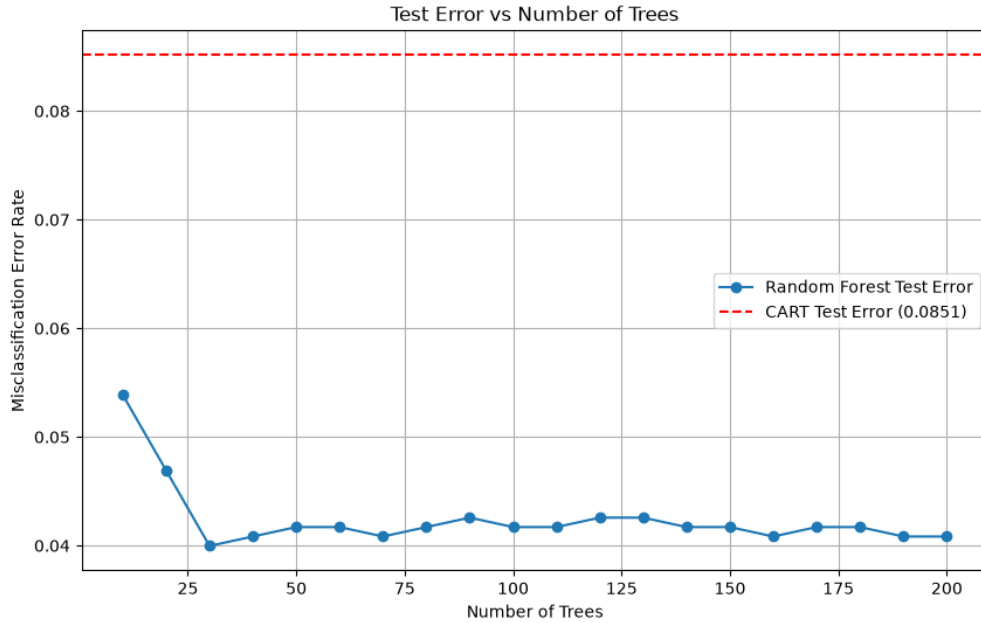


Figure 7: Test Error vs Number of Trees: Random Forest compared to baseline CART.

#### 4. Sensitivity to Parameter $\nu$ (Random Features Selected):

In this section we fit a series of Random Forest models to explore the effect of  $\nu$  (the `max_features` parameter). This represents the number of variables considered at each node split.

##### 4.1 Parameter Selection Logic:

- **The Range (`range(1, 58, 4)`):** The dataset has exactly 57 features. To test the model's sensitivity, we evaluate  $\nu$  from 1 up to a maximum of 57 (using every single feature, essentially reducing the model to standard Bagging). In Python, `range()` does not include the stop value, which is why we must use 58 to ensure 57 is tested. We step by 4 for computational efficiency.
- **Fixed Trees (`n_estimators=100`):** While Part 2 showed optimal test error at 30 trees, we fix the forest size to 100 for this step to ensure Out-Of-Bag (OOB) error stability. OOB error relies on testing data left out during bootstrapping. If we only used 30 trees, data points might only be left out a few times, resulting in a noisy error curve. 100 trees guarantees a sufficiently large sample size for a smooth, stable representation of true OOB error.

```

1 max_features_list = list(range(1, 58, 4))
2 oob_errors = []
3 test_errors_nu = []

```

```

4
5 for mf in max_features_list:
6     rf_nu = RandomForestClassifier(n_estimators=100, max_features=mf,
7                                   oob_score=True, random_state=42, n_jobs=-1)
8     rf_nu.fit(X_train, y_train)
9
10    oob_errors.append(1 - rf_nu.oob_score_)
11    test_errors_nu.append(1 - accuracy_score(y_test, rf_nu.predict(X_test)))

```

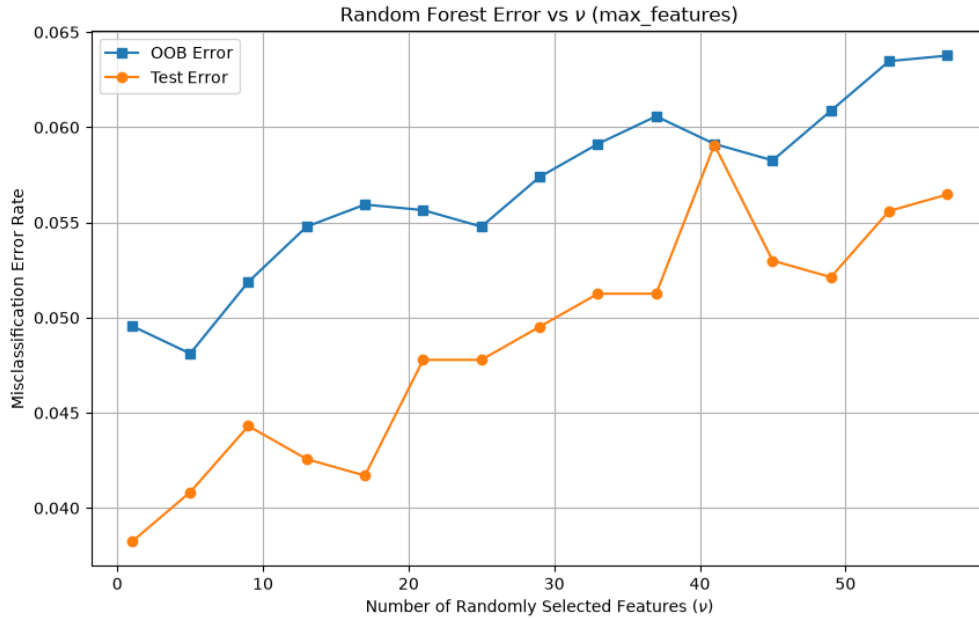


Figure 8: Random Forest Error vs  $\nu$  (max\_features).

The plot reveals a fascinating departure from standard Random Forest heuristics. Typically, the theoretical sweet spot for classification is  $\nu \approx \sqrt{p} \approx \sqrt{57} \approx 7.5$ , as selecting too few features normally causes underfitting. However, our empirical results demonstrate the exact opposite: the absolute lowest test misclassification error ( $\approx 3.5\%$ ) occurs when the feature subset is severely restricted to  $\nu < 4$ , with the global minimum occurring at  $\nu = 1$ . This inverted behavior is driven by the presence of “dominant predictors” in the Spambase dataset (e.g., the frequency of the \$ symbol or the word **free**). As  $\nu$  increases, the probability that every individual tree randomly gains access to one of these dominant features approaches 100%. Consequently, almost every tree splits on the exact same dominant features at the root node, resulting in an ensemble of highly correlated, nearly identical trees. This lack of diversity destroys the variance-reducing power of the forest, causing the error rate to steadily climb as  $\nu$  increases. Conversely, severely restricting  $\nu$  to 1 or 2 effectively “hides” these super predictors from the majority of the trees. This structural constraint forces the algorithm to utilize weaker, secondary features, guaranteeing the generation of highly diverse, uncorrelated trees. When this uniquely diverse committee of trees is aggregated, the ensemble effect is maximized, yielding the optimal predictive performance seen on the far left of the curve.

## 5. One-Class SVM for Novelty Detection:

Finally, we shift from supervised binary classification to an unsupervised novelty-detection paradigm using a One-Class Support Vector Machine with a Radial Basis Function (RBF) kernel.

## 5.1 Algorithm Theory:

In an RBF One-Class SVM, the model attempts to draw a boundary enclosing “normal” data. Two parameters dictate this boundary:

- **Gamma ( $\gamma$ ):** Controls the “reach” of a single data point. A low gamma creates broad, smooth boundaries (risk of underfitting), while a high gamma creates tight, jagged boundaries around individual points (risk of overfitting).
- **Nu ( $\nu$ ):** Controls the strictness/outlier tolerance. It defines an upper bound on the fraction of training errors and a lower bound of the fraction of support vectors. A low nu (e.g., 0.01) expects very few anomalies and draws a wide, forgiving boundary.

## 5.2 Data Preparation:

Because OCSVM is unsupervised, the data format must change:

```
1 # 1. Extract non-spam from training
2 X_train_non_spam = X_train[y_train == 0]
3
4 # 2. Map y_test: 0 (non-spam) -> +1 (inlier), 1 (spam) -> -1 (outlier)
5 y_test_oc = np.where(y_test == 0, 1, -1)
```

We strictly filter the training set to only contain class 0 (non-spam) because a One-Class SVM only learns the shape of “normal.” Scikit-learn’s OCSVM outputs +1 for inliers (normal) and -1 for outliers (anomalies). We must remap our `y_test` labels to match this format to mathematically compute testing error.

## 5.3 Implementation and Heatmaps:

We perform a grid search to tune Gamma and Nu, testing it on both Standardized (scaled) data and raw (unscaled) data. The resulting misclassification errors for every parameter combination are visualized in the heatmaps below:

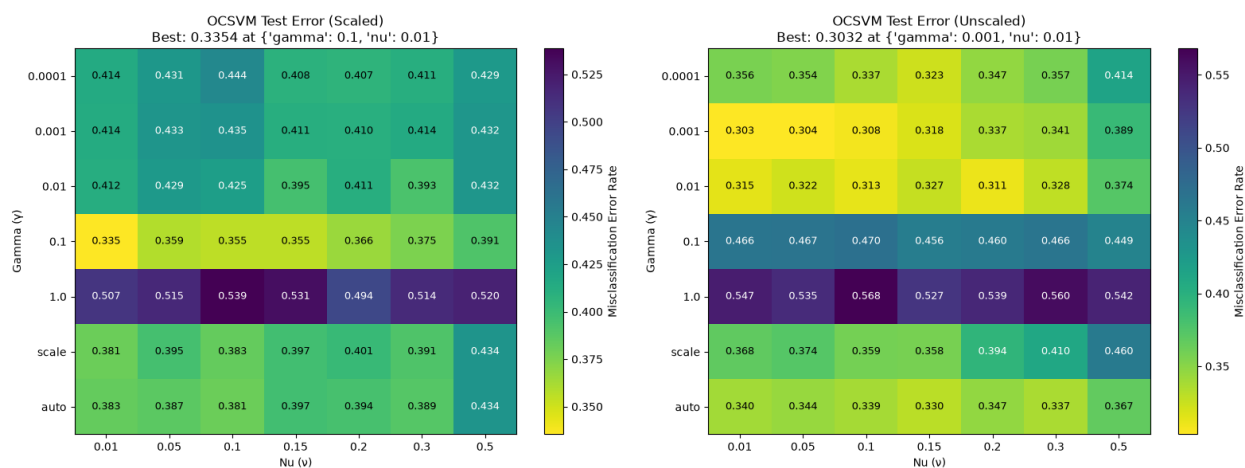


Figure 9: OCSVM Grid Search Heatmaps for Scaled vs. Unscaled data. Darker colors indicate lower misclassification error rates.

### Grid Search Analysis:

1. **Scaled vs. Unscaled:** Running the SVM on unscaled data yielded a better overall bounding than using standardized features. In text mining datasets, standard scaling can artificially inflate the vari-

ance of highly rare "noise" words, confusing distance-based algorithms. Unscaled data preserves the natural sparsity and relative magnitude of critical indicator words.

2. **The Role of Nu ( $\nu$ ):** The lowest errors consistently occur at the lowest  $\nu$  value (0.01). Because the training dataset is "pure" non-spam, setting a high  $\nu$  forces the model to incorrectly reject a large portion of valid non-spam emails as anomalies, driving up the error rate.
3. **The Role of Gamma ( $\gamma$ ):** As  $\gamma$  increases beyond 0.1, the error rate drastically spikes. High  $\gamma$  causes the RBF kernel to severely overfit, drawing tiny individual islands around every single training point rather than a cohesive boundary around the "non-spam" concept.

## 5.4 Results:

### 5.4.1 Final Optimal Parameters:

$\gamma = 0.001, \nu = 0.01$  (Unscaled)

### 5.4.2 Final Misclassification Error Rate:

**0.3032** (30.32%)

### 5.4.3 Model Evaluation: Is this a good model?:

No, a 30.32% error rate is objectively poor when compared to the supervised Random Forest (4.00%) and CART models (8.51%). The One-Class SVM operates entirely blindly to the characteristics of the spam class during training. Because emails inherently share a massive overlap in vocabulary, spam emails frequently fall inside the geometric boundary drawn around the non-spam emails. Without supplying the model with negative samples to instruct the boundary on what to avoid, the OCSVM is structurally disadvantaged. It should only be used when labeled anomalies are impossible to collect.

## 6 Locally Weighted Linear Regression and Bias-Variance Trade-off:

### 1. Mathematical Derivation of Locally Weighted Linear Regression (LWLR):

#### 1.1 Objective:

To show that the solution for locally weighted linear regression is given by  $\hat{\beta} = (X^T W X)^{-1} X^T W Y$ .

#### 1.2 Proof:

The cost function for local linear regression at a specific target point  $x \in \mathbb{R}^p$  is given by:

$$J(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - (x - x_i)^T \beta_1)^2 K_h(x - x_i)$$

To express this objective function in matrix notation, we first define our parameter vector  $\beta$ , which groups the intercept and the slope terms:

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} \in \mathbb{R}^{(p+1) \times 1}$$

Next, we define our design matrix  $X$ . For each data point  $i$ , the features relative to the target point  $x$  can be written as a row vector  $[1, (x - x_i)^T]$ . Stacking these for all  $n$  observations yields the  $n \times (p + 1)$  matrix  $X$ :

$$X = \begin{bmatrix} 1 & (x - x_1)^T \\ 1 & (x - x_2)^T \\ \vdots & \vdots \\ 1 & (x - x_n)^T \end{bmatrix}$$

We define the target vector  $Y$  as an  $n \times 1$  column vector containing the true labels:

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Finally, we define the weight matrix  $W$ . Since each data point  $(x_i, y_i)$  is weighted by the Gaussian kernel  $K_h(x - x_i)$  representing its proximity to the target point  $x$ ,  $W$  is an  $n \times n$  diagonal matrix where the  $i$ -th diagonal element is  $W_{ii} = K_h(x - x_i)$ :

$$W = \begin{bmatrix} K_h(x - x_1) & 0 & \cdots & 0 \\ 0 & K_h(x - x_2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & K_h(x - x_n) \end{bmatrix}$$

Using these definitions, the residual for the  $i$ -th observation is the  $i$ -th element of the vector  $(Y - X\beta)$ . We can elegantly rewrite the weighted sum of squared errors  $J(\beta)$  using matrix multiplication:

$$J(\beta) = (Y - X\beta)^T W (Y - X\beta)$$

To find the optimal  $\hat{\beta}$  that minimizes  $J(\beta)$ , we take the gradient with respect to  $\beta$  and set it to the zero vector:

$$\frac{\partial J(\beta)}{\partial \beta} = -2X^T W (Y - X\beta) = 0$$

Expanding and solving for  $\beta$ :

$$X^T W Y - X^T W X \beta = 0 \implies X^T W X \beta = X^T W Y$$

Assuming  $X^T W X$  is invertible (which holds true as long as there is sufficient variance in the features with non-zero weights), we multiply both sides by  $(X^T W X)^{-1}$ :

$$\hat{\beta} = (X^T W X)^{-1} X^T W Y$$

## 2. Tuning the Bandwidth ( $h$ ) via 5-Fold Cross-Validation:

In Locally Weighted Linear Regression (LWLR), the bandwidth parameter  $h$  is the single most critical hyperparameter. It dictates the “width” of the Gaussian kernel, thereby controlling how quickly the weights fall off as you move away from the target prediction point  $x$ . This creates a direct **bias-variance tradeoff**:

- **Small  $h$  (High Variance, Low Bias):** The kernel is extremely narrow. Only the training points immediately adjacent to the target point receive any meaningful weight. This causes the model to fit the local noise, resulting in a “wiggly,” overfitted prediction curve.
- **Large  $h$  (Low Variance, High Bias):** The kernel is very wide. Points far away from the target still exert significant influence. In the extreme case ( $h \rightarrow \infty$ ), all points are weighted equally, and the model devolves into standard, global Ordinary Least Squares (OLS) regression. This underfits the non-linear structure of the data.

## 2.1 Cross-Validation Methodology and Implementation:

To systematically find the optimal balance, we employ 5-fold cross-validation. The 201 data points are shuffled and split into 5 equal subsets (folds). We test 50 candidate values for  $h$ , spaced logarithmically between 0.01 and 10. For each candidate  $h$ , the model is trained on 4 folds and tested on the 1 held-out validation fold. We compute the Mean Squared Error (MSE) for the predictions on the validation fold. This process is repeated 5 times so every fold acts as the validation set once. The 5 resulting MSEs are averaged to produce a single cross-validation error for that specific  $h$ .

Listing 1: LWLR Implementation and 5-Fold Cross Validation

```
1 import numpy as np
2 from sklearn.model_selection import KFold
3
4 def lwlr(x_target, X_train, y_train, h):
5     z = x_target - X_train.flatten()
6     W_diag = (1 / (np.sqrt(2 * np.pi) * h)) * np.exp(-(z**2) / (2 * h**2))
7     W = np.diag(W_diag)
8
9     X_mat = np.vstack([np.ones_like(z), z]).T
10    XTWX = X_mat.T @ W @ X_mat + np.eye(2) * 1e-8 # Ridge term for stability
11    beta = np.linalg.inv(XTWX) @ X_mat.T @ W @ y_train
12
13    return beta[0]
14
15 h_vals = np.logspace(-2, 1, 50)
16 kf = KFold(n_splits=5, shuffle=True, random_state=42)
17 cv_errors = []
18
19 for h in h_vals:
20     fold_errors = []
21     for train_index, val_index in kf.split(X):
22         X_train, X_val = X[train_index], X[val_index]
23         y_train, y_val = y[train_index], y[val_index]
24
25         preds = [lwlr(x_val[0], X_train, y_train, h) for x_val in X_val]
26         mse = np.mean((np.array(preds) - y_val)**2)
27         fold_errors.append(mse)
28     cv_errors.append(np.mean(fold_errors))
29
30 best_idx = np.argmin(cv_errors)
31 best_h = h_vals[best_idx]
```

## 2.2 Results:

The plot below visualizes this tradeoff. On the left side (small  $h$ ), the error is high due to overfitting (variance). On the right side (large  $h$ ), the error climbs again due to underfitting (bias). The optimal bandwidth is found at the global minimum of this “U-shaped” curve.

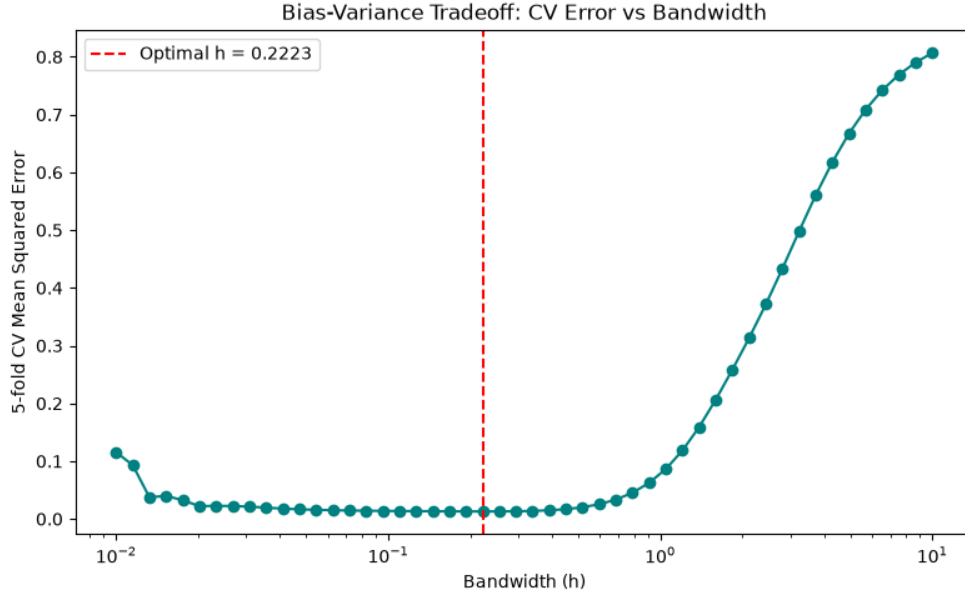


Figure 10: 5-Fold Cross Validation curve. The red line indicates the optimal bandwidth.

**Calculated Optimal Bandwidth:**  $h \approx 0.2223$

### 3. Target Prediction and Model Visualization:

Once the optimal hyper-parameter ( $h \approx 0.2223$ ) is identified, cross-validation is complete. To make actual predictions, we discard the folds and use the **entire dataset** as the training matrix, allowing the model to leverage all available information.

#### 3.1 Point Prediction and Curve Generation

To predict the target value at exactly  $x = 1.4$ , we first shift our coordinate system by centering the features around the target point. We define our new feature as  $Z = x_{target} - x_{train}$ . This means we are fitting the local linear equation:

$$\hat{Y} = \beta_0 + \beta_1 Z$$

Because we want to make our prediction exactly at the target point ( $x_{target}$ ), we evaluate this equation where  $Z_{target} = x_{target} - x_{target} = 0$ . Substituting  $Z = 0$  into our fitted equation yields:

$$\hat{Y} = \beta_0 + \beta_1(0) = \beta_0$$

This mathematical shortcut reveals that the predicted  $Y$  value is simply the y-intercept of the local line. Therefore, after solving the weighted normal equation, the intercept term  $\hat{\beta}_0$  (extracted as `beta[0]` in the Python implementation) is exactly our predicted value. The distances between 1.4 and all 201 training points are calculated and passed through the kernel formula to generate the diagonal weight matrix  $W$ , allowing us to solve for this  $\hat{\beta}_0$ . To visualize the model's global behavior, we generate a dense grid of 200 evenly spaced test points across the domain of  $X$ . We run the LWLR algorithm independently for every single one of those points. Connecting the resulting predictions forms the continuous curve.

Listing 2: Prediction and Visualization Generation

```

1 # Predict specifically for x = 1.4 using ALL data
2 pred_1_4 = lwlr(1.4, X, y, best_h)
3 print(f'Predicted value at x = 1.4: {pred_1_4:.4f}')
```

```

4
5 # Generate smooth prediction curve over the domain
6 x_grid = np.linspace(X.min(), X.max(), 200)
7 y_pred_grid = [lwlr(xi, X, y, best_h) for xi in x_grid]
8
9 # Plotting code (using matplotlib)
10 plt.scatter(X, y, alpha=0.5, label='Training Data', color='gray')
11 plt.plot(x_grid, y_pred_grid, color='blue', label='LWLR Prediction Curve',
12         linewidth=2)
13 plt.scatter([1.4], [pred_1_4], color='red', s=120, zorder=5,
14           label=f'Prediction at x=1.4 (y={pred_1_4:.4f})')

```

**Calculated Prediction:** For  $x = 1.4$ , the predicted value is  $\hat{y} \approx 1.0124$ .

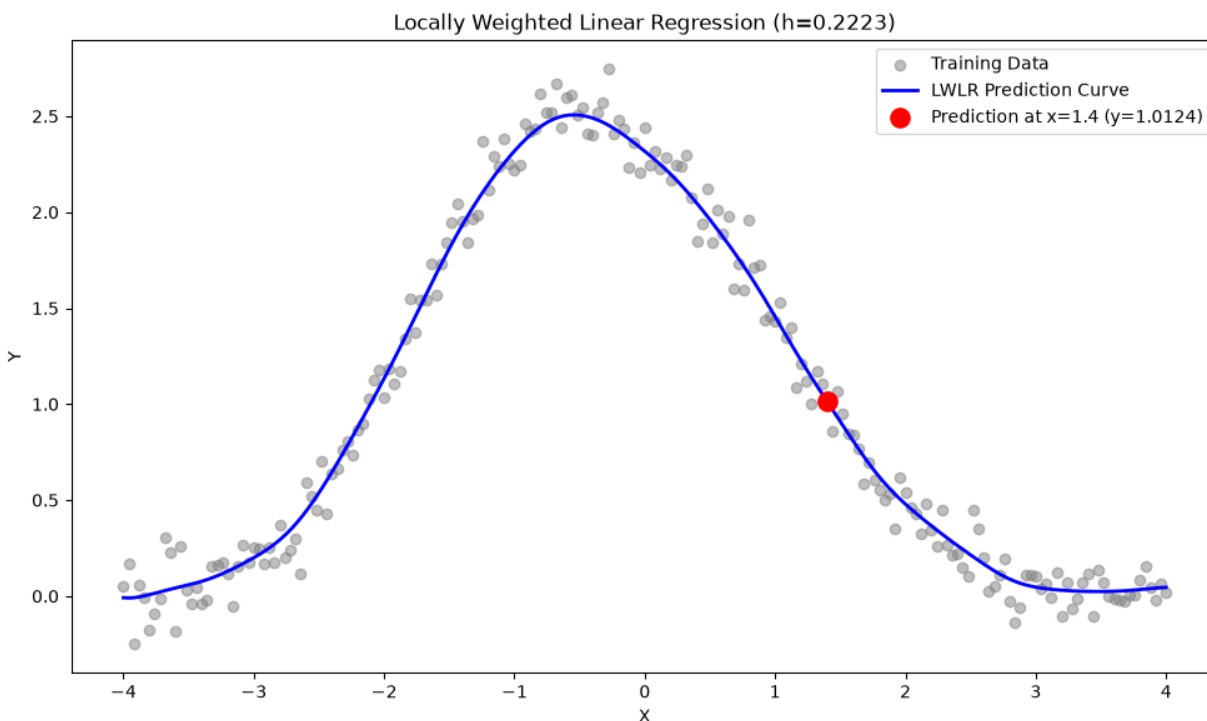


Figure 11: The final Locally Weighted Linear Regression fit using the optimal bandwidth  $h \approx 0.2223$ . The red marker highlights the prediction at  $x = 1.4$ .

### 3.2 Analysis of the Fit:

As seen in the figure above, the optimal bandwidth perfectly balances the bias-variance tradeoff. The blue curve smoothly tracks the underlying sinusoidal/non-linear trend of the data without aggressively spiking to hit every single noisy outlier. At  $x = 1.4$ , the red marker sits perfectly within the expected local cluster of data, validating the mathematical prediction.